

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Data Interpretation

Course Code:	ITA05	Course Title:	Computer Vision
CO Assessed:	CO4 – Articulation (combining methods for evaluation) P4	Assessment:	Assessment Tool 1 – Data Interpretation
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Compare object and pattern recognition techniques for interpreting visual data with spatial and motion characteristics. (BTL5)		
Student Name:	Malli Chandana	Reg. No.:	192472250
Section / Batch:	CSE-AI/2024-2028	Date:	17-08-26

Code of Conduct

I Malli Chandana(192472250)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Malli Chandana

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided	
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data	
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions	
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	

1. Feature Detection under Illumination Changes

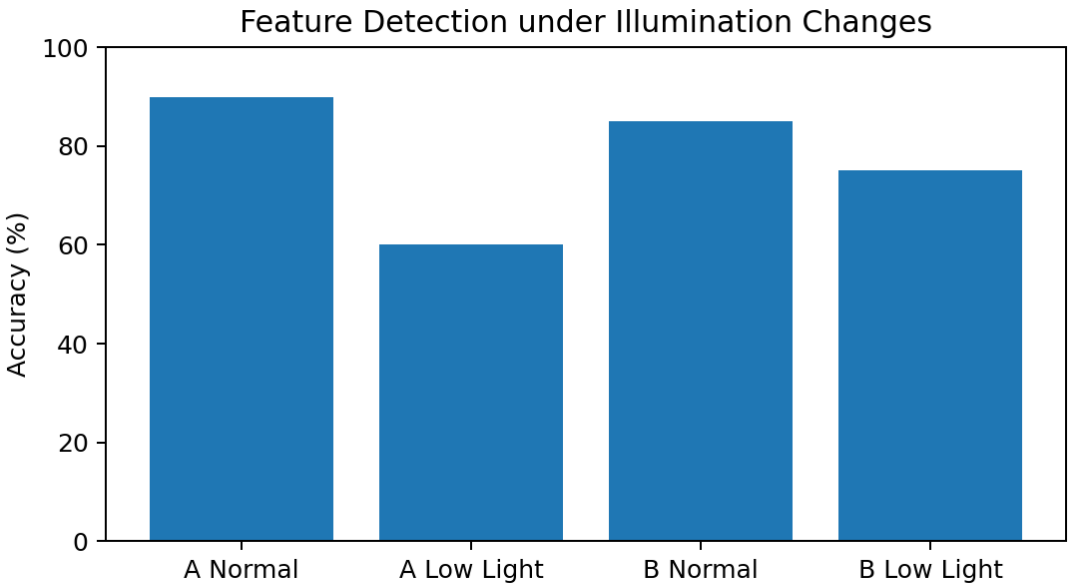
Feature detection is a fundamental computer vision task used to identify distinctive points, edges, corners, and patterns in an image. It supports applications such as object recognition, image matching, tracking, surveillance, robotics, and autonomous navigation. A major challenge is illumination change because reduced brightness can lower contrast and make important image features difficult to detect.

Method	Normal Light	Low Light
A	90%	60%
B	85%	75%

Comparison of Results

Method A gives the highest accuracy under normal lighting at 90%. However, its accuracy falls to 60% in low light, a reduction of 30 percentage points. This large drop shows that Method A is sensitive to illumination changes.

Method B gives 85% accuracy in normal lighting and 75% in low light. Its reduction is only 10 percentage points. Although it is 5 percentage points below Method A in normal conditions, it performs 15 percentage points better than Method A in low light.



Robustness Evaluation

Robustness refers to the ability of a technique to maintain performance when operating conditions change. Method B is more robust because its performance remains relatively stable between normal and low-light conditions. Its average accuracy is 80%, compared with 75% for Method A.

The supplied data does not provide processing time, memory use, or hardware requirements, so computational superiority cannot be claimed. If both methods have similar computational requirements, Method B is preferable for variable lighting.

Application Context and Decision

Method A is suitable for controlled environments such as laboratories, factories, and well-lit indoor systems where illumination is stable. Method B is better for night-time surveillance, outdoor robotics, mobile vision, and autonomous vehicles, where lighting may change unexpectedly.

Therefore, Method B is the recommended overall method when illumination conditions are unpredictable. Method A should be selected only when high accuracy in normal lighting is the main requirement.

2. Video Processing Pipeline Comparison

A video processing pipeline contains stages such as image acquisition, preprocessing, feature extraction, object detection, tracking, classification, and output generation. The best pipeline depends on both accuracy and processing speed. In real-time computer vision, a highly accurate but slow pipeline may be less useful than a slightly less accurate pipeline that responds quickly.

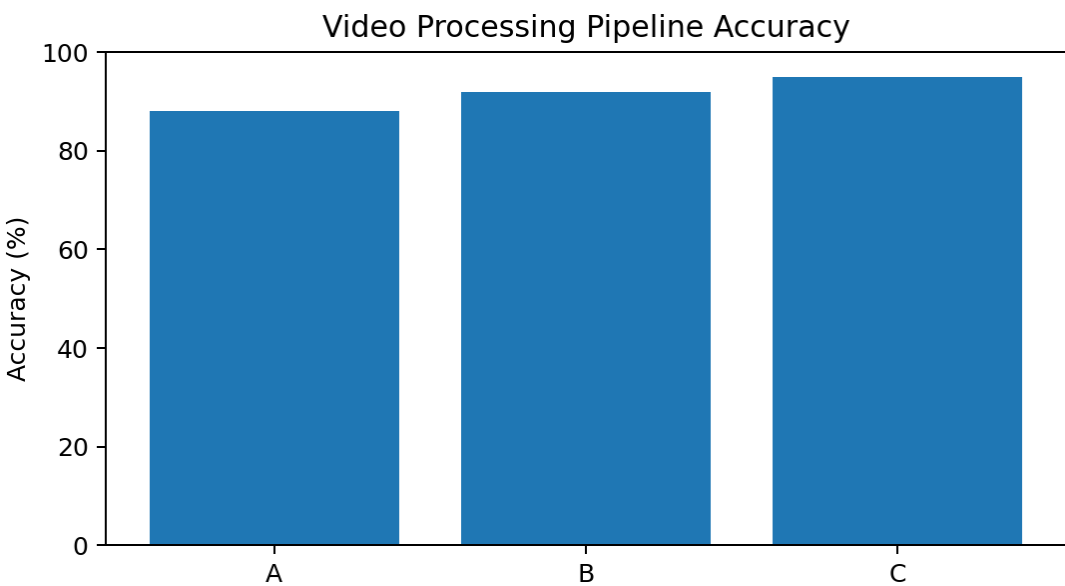
Pipeline	Accuracy	Speed
A	88%	Fast
B	92%	Moderate
C	95%	Slow

Evaluation of Pipeline A

Pipeline A has the lowest accuracy at 88%, but it is fast. Its main advantage is rapid processing, making it suitable when immediate results are important. Examples include real-time CCTV, traffic monitoring, embedded vision, and mobile applications.

Evaluation of Pipeline B

Pipeline B achieves 92% accuracy with moderate speed. It improves accuracy by 4 percentage points over Pipeline A while avoiding the slow processing of Pipeline C. This gives it a useful balance between accuracy and speed.



Evaluation of Pipeline C

Pipeline C achieves the highest accuracy of 95%, but its speed is slow. It improves accuracy by 7 percentage points over Pipeline A and 3 percentage points over Pipeline B. The extra accuracy can be valuable when incorrect detections are costly, but the slow speed limits real-time use.

Suitability and Decision

Pipeline A is appropriate when speed is the dominant requirement. Pipeline C is better for offline video analysis, forensic investigation, or research where processing time is less important. Pipeline B is the best general-purpose choice because it offers 92% accuracy with moderate speed.

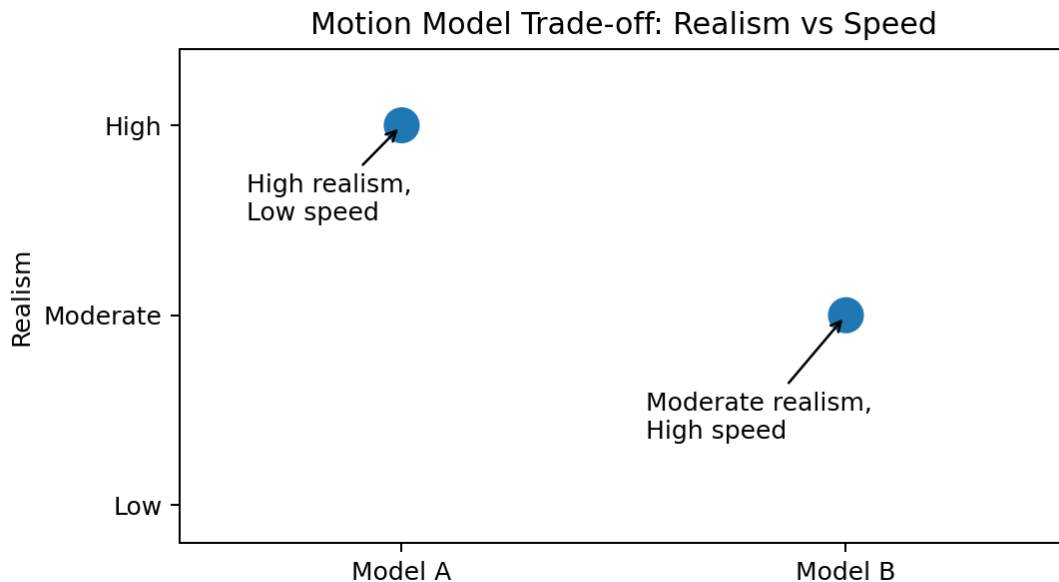
For applications such as intelligent traffic monitoring or robotics, Pipeline B provides a practical compromise. For strict real-time systems, Pipeline A may be preferred, while high-accuracy offline analysis favors Pipeline C.

Overall, Pipeline B is recommended because it provides the most balanced combination of accuracy and processing speed.

3. Motion Model Evaluation

Motion modelling describes how an object changes position over time and is widely used in object tracking, robotics, autonomous vehicles, surveillance, animation, gaming, virtual reality, and simulation. The important factors in this comparison are realism and speed.

Model	Realism	Speed
A	High	Low
B	Moderate	High



Evaluation of Model A

Model A provides high realism but low speed. It can represent complex movement more accurately and is suitable when realistic physical behaviour is more important than immediate processing. Examples include film animation, visual effects, detailed simulation, and offline motion analysis.

The main limitation is slow processing. A real-time system may not be able to wait for detailed calculations before updating the object's movement.

Evaluation of Model B

Model B provides moderate realism but high speed. It is computationally efficient and can update motion quickly. This makes it suitable for robotics, real-time surveillance, interactive gaming, and autonomous systems.

Its limitation is lower realism. It may simplify complex movement, so it is less suitable when the visual or physical accuracy of motion is the primary requirement.

Application-Based Decision

For animation and detailed simulation, Model A is preferred because realism is more important than response time. For robotics, autonomous vehicles, and real-time surveillance, Model B is preferred because fast response is essential.

There is therefore no universally superior model. The correct choice depends on the application requirement. If realism is the priority, choose Model A; if speed and real-time response are the priority, choose Model B.

Conclusion

Model A is the better choice for high-realism and offline applications, whereas Model B is better for fast, real-time applications. The trade-off between realism and speed should guide the final selection.

4. Detection under Occlusion

Occlusion occurs when an object is partially or completely hidden by another object. It is a major challenge in object detection because the system cannot observe the complete target. Occlusion is common in crowded roads, surveillance scenes, pedestrian detection, robotics, and autonomous vehicles.

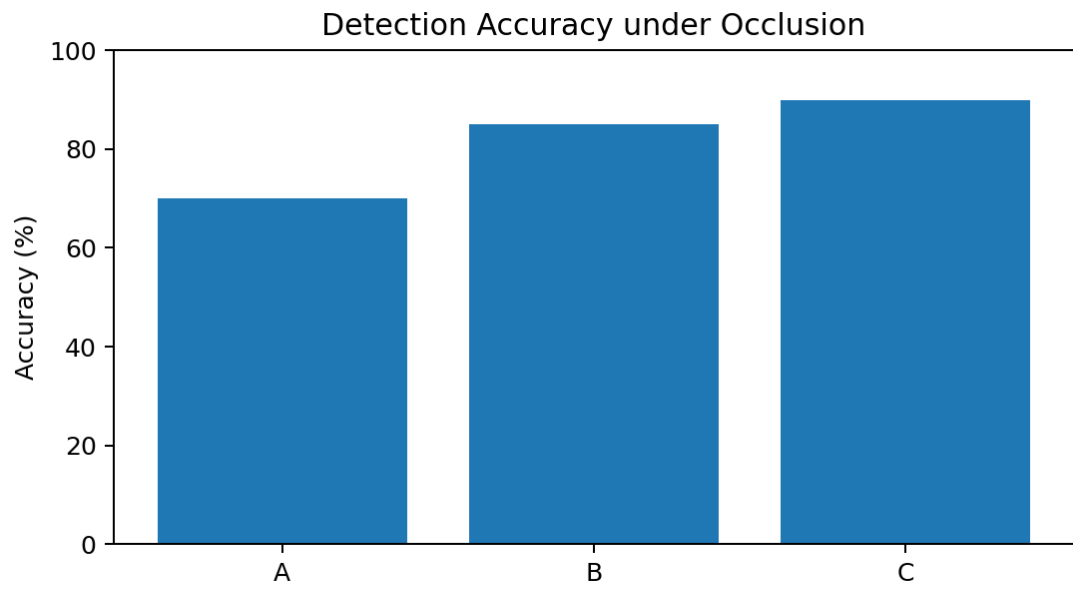
Method	Accuracy
A	70%
B	85%
C	90%

Comparison of Methods

Method A achieves 70% accuracy, the lowest among the three methods. This indicates weak robustness under occlusion and suggests that many partially hidden objects may be missed or incorrectly detected.

Method B achieves 85% accuracy, which is 15 percentage points higher than Method A. It therefore handles partially hidden objects more effectively and provides good robustness for moderately complex environments.

Method C achieves the highest accuracy of 90%. It improves on Method A by 20 percentage points and on Method B by 5 percentage points. Therefore, Method C is the most robust method under the given conditions.



Application Context

Method A may be acceptable in simple environments where objects are mostly visible. Method B is suitable for general surveillance and traffic monitoring where some overlap occurs. Method C is preferable for crowded and safety-critical environments such as autonomous vehicles, busy roads, airports, and pedestrian monitoring.

Computational Consideration

The given data contains only accuracy values. It does not specify latency, memory use, or computational cost. Therefore, computational efficiency cannot be compared directly. If Method C requires substantially more resources, Method B could be considered as a practical alternative.

Conclusion

Based on the available results, Method C is the best choice for robustness under occlusion because it achieves the highest accuracy of 90%. Method B is a good alternative, while Method A is least suitable for heavily occluded scenes.

5. System Performance under Constraints

A practical computer vision system should not be selected using accuracy alone. Accuracy, latency, and cost must be considered together. Accuracy measures correctness, latency represents response time, and cost reflects the resources required to implement and operate the system.

Model	Accuracy	Latency	Cost
A	88%	50 ms	Low
B	92%	80 ms	Moderate
C	95%	120 ms	High

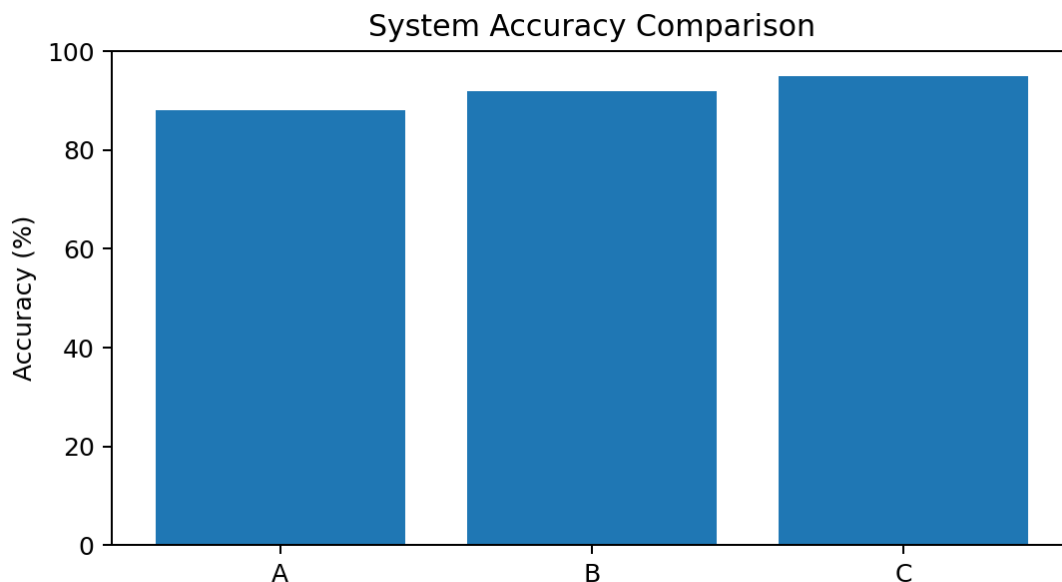
Evaluation of Model A

Model A has the lowest accuracy at 88%, but it also has the lowest latency of 50 ms and low cost. Its fast response makes it suitable for real-time monitoring, embedded systems, mobile applications, and budget-constrained deployments.

Its limitation is lower accuracy. If the application can tolerate some reduction in detection or prediction accuracy, Model A provides a fast and economical solution.

Evaluation of Model B

Model B achieves 92% accuracy with 80 ms latency and moderate cost. It improves accuracy by 4 percentage points over Model A while adding 30 ms of latency. Compared with Model C, it is less accurate but faster and less costly.



Evaluation of Model C

Model C gives the highest accuracy at 95%, but it has the highest latency of 120 ms and high cost. Its accuracy advantage over Model B is only 3 percentage points, while the increase in latency and cost is substantial.

Priority	Recommended Model	Reason
Lowest latency / cost	A	50 ms and low cost
Maximum accuracy	C	95% accuracy
Balanced performance	B	92% accuracy, moderate latency and cost

Final Decision

For a general-purpose system where accuracy, response time, and cost all matter, Model B is the best overall choice. It avoids the lower accuracy of Model A and the high latency and cost of Model C.

Model A should be selected when low cost and minimum latency are critical. Model C should be selected when maximum accuracy is worth the additional cost and delay. Therefore, considering all constraints together, Model B provides the most practical trade-off.

Overall Conclusion

The comparison demonstrates that the highest-accuracy technique is not always the best practical system. Model B provides a balanced solution and is recommended for most general applications. The final selection should always be based on the specific requirements and constraints of the application.